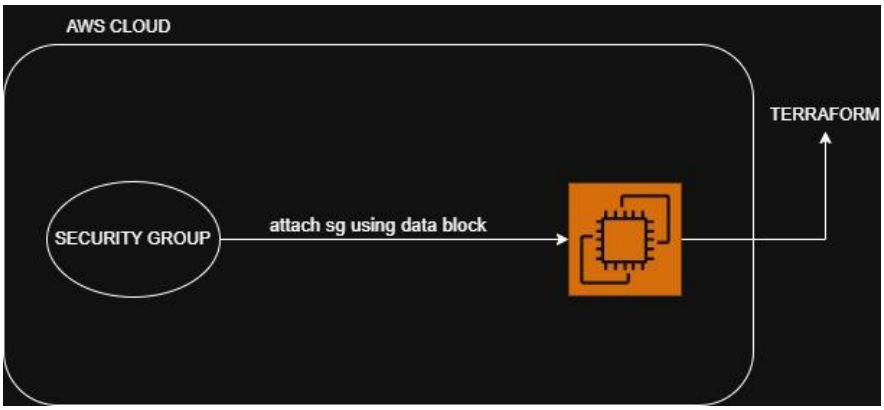
DEPLOYING LOAD BALANCER AND AUTO SCALING WITH TERRAFORM

DATA BLOCK

a data block is used to read information about existing resources without creating or modifying them.



1.provider.tf

```
provider "aws" {
   region = "ap-south-1"
   profile = "config"
}
```

2.resource.tf

```
data "aws_security_group" "web_sg" {
   name = "my-sg"
}

resource "aws_instance" "my_instance" {
  ami                    = "ami-0685bcc683dadb6b9"
  instance_type       = "t3.micro"
  key_name             = "ubuntu"
  vpc_security_group_ids = [data.aws_security_group.web_sg.id]

  tags = {
    Name = "webserver"
  }
}
```

3.git add .
4.git commit -m “resource.tf”

5.git push

ON UBUNTU LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

VARIABLE BLOCK

A variable block in Terraform is used to make your code dynamic and reusable instead of hardcoding values.

1.provider.tf

```
provider "aws" {
   region = "ap-south-1"
   profile = "config"
}
```

2.resource.tf

```
resource "aws_instance" "my_instance" {
 ami                    = var.ami
 instance_type          = var.instance_type
 key_name               = var.key_name
 vpc_security_group_ids = [var.sg]
 count                  = var.counts
 disable_api_termination = var.api_termination
 tags                   = var.tags
}
```

3.variable.tf

```
variable "ami" {
   default = "ami-0685bcc683dadb6b9"
}

variable "instance_type" {
   default = "t3.micro"
}
variable "key_name" {
   default = "ubuntu"
```

```
}

variable "sg" {
   default = "sg-0d753ab93387e2811"
}

variable "counts" {
   default = 2
}

variable "api_termination" {
   default = true
}

variable "tags" {
  type = map(string)
  default = {
    Name        = "webserver"
    Environment = "dev"
    Owner       = "Mayur"
  }
}
```

3.git add .
4.git commit -m “variable.tf”
5.git push

ON LOCAL UBUNTU

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

OUTPUT BLOCK

In Terraform, an Output Block is used to display values after Terraform creates or updates infrastructure.

It helps in

- View important resource information.
- Pass values to other Terraform configurations.
- Avoid manually searching for resource details.

1.provider.tf

```
provider "aws" {
   region = "ap-south-1"
   profile = "config"
}
```

2.resource.tf

```
resource "aws_instance" "my_instance" {
  ami                    = var.ami
  instance_type          = var.instance_type
  key_name               = var.key_name
  vpc_security_group_ids = [var.sg]
  disable_api_termination = var.api_termination
  tags                           = var.tags
}
```

3.variable.tf

```
variable "ami" {
   default = "ami-0685bcc683dadb6b9"
}

variable "instance_type" {
   default = "t3.micro"
}

variable "key_name" {
   default = "ubuntu"
}

variable "sg" {
   default = "sg-0d753ab93387e2811"
}
```

```
variable "counts" {
   default = 2
}

variable "api_termination" {
   default = true
}

variable "tags" {
  type = map(string)
  default = {
    Name        = "webserver"
    Environment = "dev"
    Owner       = "Mayur"
  }
}
```

4.output.tf

```
output "public_ip" {
   value = aws_instance.my_instance.public_ip
}

output "public_dns" {
   value = aws_instance.my_instance.public_dns
}

output "instance_id" {
   value = aws_instance.my_instance.id
}

output "private_ip" {
   value = aws_instance.my_instance.private_ip
}
```

5.git add .
6.git commit -m “terraform-block”
7.git push

ON UBUNTU LOCAL

1. git clone
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve

5. terraform destroy –auto-approve

HOW TO CREATE CUSTOM KEYS & ASSIGN TO A INSTANCE

ON UBUNTU-LOCAL

1.ssh-keygen
2.cd .ssh/
3.ls
 id_rsa id_rsa.pub

2.resource.tf

```
resource "aws_key_pair" "my-key" {
 key_name   = "my-key"
 public_key = file("~/.ssh/id_rsa.pub")
}

resource "aws_instance" "my_instance" {
 ami                    = “ami-0685bcc683dadb6b9”
 instance_type          = “t3.micro”
 key_name               = aws_key_pair.my-key.key_name
 tags = {
   Name = “my-server”
  }
}
```

1. git add .
2. git commit -m “keys”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

CREATION OF SECURITY GROUP & ASSIGN TO INSTANCE

1.security-group.tf

```
data "aws_vpc" "my_vpc" {
   default = true
}

resource "aws_security_group" "my_sg" {
   name          = "my-security-group"
   description = "my-sg"
   vpc_id        = data.aws_vpc.my_vpc.id

   ingress {
      from_port = 80
      to_port      = 80
      protocol     = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
   }

   ingress {
      from_port = 22
      to_port      = 22
      protocol     = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
   }

   egress {
      from_port = 0
      to_port      = 0
      protocol     = "-1"
      cidr_blocks = ["0.0.0.0/0"]
   }

   tags = {
      Name = "my-security-group"
   }
}
```

2.ec2.tf

```
resource "aws_instance" "my_instance" {
  ami                  = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
```

```
  key_name                    = "ubuntu"
  vpc_security_group_ids = [aws_security_group.my_sg.id]
  user_data                   = file("/root/terraform-b30/user-data.sh")

  root_block_device {
    volume_size   = 20
    volume_type = "gp3"
  }

  tags = {
    Name = "my-instance"
  }
}
```

3.user-data.sh

```
#!/bin/bash
dnf update -y
dnf install httpd -y
systemctl enable httpd
systemctl start httpd
echo "<h1> Welcome from Terraform </h1>" > /var/www/html/index.html
```

1. git add .
2. git commit -m “resources”
3. git push

ON UBUNTU LOCAL

1. git clone
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

DEPLOYING LOAD BALANCER AND AUTO SCALING WITH TERRAFORM

1.main.tf

```
data "aws_vpc" "default" {
 default = true
}

data "aws_subnets" "public" {
 filter {
  name   = "vpc-id"
  values = [data.aws_vpc.default.id]
 }
}
```

#CREATE A SECURITY-GROUP

```
resource "aws_security_group" "alb_sg" {
  name = "alb_sg"
  description = "my-sg"
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "my-security-group"
```

```
  }
}

#CREATE A LAUNCH TEMPLATE
resource "aws_launch_template" "web" {
 name_prefix   = "web-template"
 image_id        = "ami-0685bcc683dadb6b9"
 instance_type = "t3.micro"
 key_name       = "ubuntu"
 vpc_security_group_ids = [aws_security_group.alb_sg.id]
 user_data             = filebase64("/root/terraform-b30/user-data.sh")
}

#CREATE A TARGET GROUP
resource "aws_lb_target_group" "tg" {
 name     = "web-tg"
 port      = 80
 protocol = "HTTP"
 vpc_id   = data.aws_vpc.default.id
 health_check {
  path = "/"
 }
}

#CREATE A APPLICATION LOAD BALANCER
resource "aws_lb" "alb" {
 name               = "my-alb"
 internal           = false
 load_balancer_type = "application"
 security_groups = [aws_security_group.alb_sg.id]
 subnets = [
  "subnet-05645f5ec7e94c61b",
  "subnet-00e4fff967bd0cf50"
 ]
}

#CREATE A LISTENER
resource "aws_lb_listener" "listener" {
 load_balancer_arn = aws_lb.alb.arn
 port     = 80
```

```
  protocol = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}
```

#CREATE A AUTOSCALING GROUP

```
resource "aws_autoscaling_group" "asg" {
  name = "web-asg"
  desired_capacity = 2
  min_size = 2
  max_size = 4
  vpc_zone_identifier = [
    "subnet-05645f5ec7e94c61b",
    "subnet-00e4fff967bd0cf50"
  ]
  target_group_arns = [aws_lb_target_group.tg.arn]
  launch_template {
    id          = aws_launch_template.web.id
    version = "$Latest"
  }
  health_check_type = "ELB"
}
```

2.user-data.sh

```
#!/bin/bash
dnf update -y
dnf install httpd -y
systemctl enable httpd
systemctl start httpd
echo "<h1> Welcome from Terraform $HOSTNAME </h1>" >
/var/www/html/index.html
```

3.output.tf

```
output "alb_dns_name" {
  value = aws_lb.alb.dns_name
}
```

1. git add .

2. git commit -m “alb-asg”
3. git push

ON UBUNTU LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

CREATING VPC WITH TERRAFORM

1.main.tf

```
resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "my-vpc"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.0.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet"
  }
}

resource "aws_subnet" "private_subnet" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.1.0/24"
  availability_zone = "ap-south-1b"
  tags = {
    Name = "private-subnet"
  }
}
```

```
resource "aws_internet_gateway" "IGW" {
  vpc_id = aws_vpc.my_vpc.id
  tags = {
    Name = "IGW"
  }
}

resource "aws_eip" "nat_eip" {
  domain = "vpc"
  tags = {
    Name = "nat-eip"
  }
}

resource "aws_nat_gateway" "nat_gateway" {
  subnet_id = aws_subnet.public_subnet.id
  allocation_id = aws_eip.nat_eip.id
  tags = {
    Name = "nat-gateway"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.my_vpc.i
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.IGW.id
  }
  tags = {
    Name = "public-rt"
  }
}

resource "aws_route_table_association" "public_rt_assoc" {
  subnet_id = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.my_vpc.id
```

```
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat_gateway.id
  }
  tags = {
    Name = "private-rt"
  }
}

resource "aws_route_table_association" "private_rt_assoc" {
  subnet_id = aws_subnet.private_subnet.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_security_group" "sg" {
  name = "my-security-group"
  description = "my-security-group"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  tags = {
    Name = "my-security-group"
  }
}

resource "aws_instance" "public_instance" {
  ami = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
  key_name = "ubuntu"
  subnet_id = aws_subnet.public_subnet.id
  vpc_security_group_ids = [aws_security_group.sg.id]
  associate_public_ip_address = true
  user_data = file("/root/terraform-b30/user-data.sh")
  tags = {
    Name = "public-instance"
  }
}

resource "aws_instance" "private_instance" {
  ami = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
  key_name = "ubuntu"
  subnet_id = aws_subnet.private_subnet.id
  vpc_security_group_ids = [aws_security_group.sg.id]
  user_data = file("/root/terraform-b30/user-data.sh")
  tags = {
    Name = "private-instance"
  }
}
```

2.output.tf

```
output "public_ip" {
  value = aws_instance.public_instance.public_ip
}
```

3.user-data.sh

```
#!/bin/bash
dnf update -y
dnf install httpd -y
systemctl enable httpd
```

```
systemctl start httpd
echo "<h1> Welcome from Terraform $HOSTNAME </h1>" >
/var/www/html/index.html
```

1. git add .
2. git commit -m “main”
3. git push

ON UBUNTU-LOCAL

1. git clone
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve

DEPLOYING ALB & ASG & VPC WITH TERRAFORM

1.main.tf

```
resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "my-vpc"
  }
}

resource "aws_subnet" "public_subnet-1a" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.0.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet-1a"
  }
}

resource "aws_subnet" "public_subnet-1b" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.1.0/24"
  availability_zone = "ap-south-1b"
```

```
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet-1b"
  }
}

resource "aws_subnet" "private_subnet-1a" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.2.0/24"
  availability_zone = "ap-south-1a"
  tags = {
    Name = "private-subnet-1a"
  }
}

resource "aws_subnet" "private_subnet-1b" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = "10.0.3.0/24"
  availability_zone = "ap-south-1b"
  tags = {
    Name = "private-subnet-1b"
  }
}

resource "aws_internet_gateway" "IGW" {
  vpc_id = aws_vpc.my_vpc.id
  tags = {
    Name = "IGW"
  }
}

resource "aws_eip" "nat_eip" {
  domain = "vpc"
  tags = {
    Name = "nat-eip"
  }
}

resource "aws_nat_gateway" "my_nat" {
  subnet_id = aws_subnet.public_subnet-1a.id
```

```
  allocation_id = aws_eip.nat_eip.id
  tags = {
    Name = "my-nat"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    gateway_id = aws_internet_gateway.IGW.id
    cidr_block = "0.0.0.0/0"
  }
  tags = {
    Name = "public-rt"
  }
}

resource "aws_route_table_association" "public_rt_assoc-1a" {
  subnet_id = aws_subnet.public_subnet-1a.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "public_rt_assoc-1b" {
  subnet_id = aws_subnet.public_subnet-1b.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.my_nat.id
  }
  tags = {
    Name = "private-rt"
  }
}

resource "aws_route_table_association" "private_rt_assoc-1a" {
  subnet_id = aws_subnet.private_subnet-1a.id
```

```
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_route_table_association" "private_rt_assoc-1b" {
  subnet_id = aws_subnet.private_subnet-1b.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_security_group" "sg" {
  name = "my-security-name"
  description = "my-security-group"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "my-security-group"
  }
}

resource "aws_lb_target_group" "tg" {
```

```
  name = "tg"
  port = 80
  protocol = "HTTP"
  vpc_id = aws_vpc.my_vpc.id
  health_check {
    path = "/"
  }
}

resource "aws_lb" "alb" {
  name = "alb"
  internal = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.sg.id]
  subnets = [aws_subnet.public_subnet-1a.id,aws_subnet.public_subnet-1b.id]
}

resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.alb.arn
  port     = 80
  protocol = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}

resource "aws_launch_template" "web" {
  name_prefix    = "web-template"
  image_id       = "ami-0685bcc683dadb6b9"
  instance_type = "t3.micro"
  key_name       = "ubuntu"
  vpc_security_group_ids = [aws_security_group.sg.id]
  user_data              = filebase64("/root/terraform-b30/user-data.sh")
}

resource "aws_autoscaling_group" "asg" {
  name = "web-asg"
  desired_capacity = 2
  min_size = 2
```

```
  max_size = 4
  vpc_zone_identifier = [aws_subnet.private_subnet-
1a.id,aws_subnet.private_subnet-1b.id]
  target_group_arns = [aws_lb_target_group.tg.arn]
  launch_template {
    id          = aws_launch_template.web.id
    version = "$Latest"
  }
  health_check_type = "ELB"
}
```

2.output.tf

```
output "lb_dns" {
  value = aws_lb.alb.dns_name
}
```

1. git add .
2. git commit -m “resource”
3. git push

ON UBUNTU LOCAL

1. git clone
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

STATIC WEBSITE HOSTING USING TERRAFORM

```
s3-static-website/
 │
 ├── provider.tf
 ├── variables.tf
 ├── main.tf
 ├── outputs.tf
 └── website/
       ├── index.html
       └── style.css
```

1. provider.tf

```
provider "aws" {
  region = "us-east-2"
  profile = “dev”
}
```

2. variable.tf

```
variable "bucket_name" {
  default = "mayur-static-site-demo-12345"
}
```

3. main.tf

```
resource "aws_s3_bucket" "website" {
  bucket = var.bucket_name
}

resource "aws_s3_bucket_public_access_block" "website" {
  bucket = aws_s3_bucket.website.id
  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}

resource "aws_s3_bucket_policy" "website" {
  bucket = aws_s3_bucket.website.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid = "PublicRead"
        Effect = "Allow"
        Principal = "*"
        Action = "s3:GetObject"
        Resource = "${aws_s3_bucket.website.arn}/*"
      }
    ]
  })
  depends_on = [
    aws_s3_bucket_public_access_block.website
```

```
  ]
}

resource "aws_s3_bucket_website_configuration" "website" {
  bucket = aws_s3_bucket.website.id
  index_document {
    suffix = "index.html"
  }
  error_document {
    key = "index.html"
  }
}

resource "aws_s3_object" "index" {
  bucket = aws_s3_bucket.website.id
  key = "index.html"
  source = "website/index.html"
  content_type = "text/html"
  etag = filemd5("website/index.html")
}

resource "aws_s3_object" "style" {
  bucket = aws_s3_bucket.website.id
  key = "style.css"
  source = "website/style.css"
  content_type = "text/css"
  etag = filemd5("website/style.css")
}
```

4. output.tf

```
output "website_url" {
  value = aws_s3_bucket_website_configuration.website.website_endpoint
}
```

5. mkdir website
6. vim index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Terraform Website</title>
```

```
   <link rel="stylesheet" href="style.css">
</head>
<body>

<h1>Welcome to Static Website Hosting using Terraform</h1>

<p>This website is hosted on Amazon S3.</p>

</body>
</html>
```

7. vim style.css

```
body{
   background:#f2f2f2;
   text-align:center;
   font-family:Arial;
}

h1{
   color:blue;
}

p{
   font-size:22px;
}
```

8. terraform init
9. terrform validate
10. terraform apply --auto-approve
11. terraform destroy --auto-approve
